

WEEK-2 PRACTICAL

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID

CODE:

```
GNU nano 7.2 command_executor.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%99s", command);

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: %s\n", command);

        execlp(command, command, NULL);

        printf("Command execution failed!\n");
    }

    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

OUTPUT:

```
root@Mani:~# nano command_executor.c
root@Mani:~# gcc command_executor.c -o command_executor
root@Mani:~# ./command_executor
Enter a Linux command: ls
Parent Process
Parent PID : 711
Child PID : 714

Child Process
Child PID : 714
Parent PID: 711

Executing command: ls
BufferSize.c.save  FileManager  Makefile  PROCESSTASK.save  SystemCallParent.c  command_executor  filecopy.c  obj  shellcommand tests
DisplayPID         ForkProgram.c    OSSP      README.md         SystemCallParent.c.save  command_executor.c  filecopy.c.save  process  shellcommand.c
DisplayPID.c       ForkProgram.c    OSSP_old  SystemCall.c      assets               copyfile.c          process.c         snap
File1.txt          LICENSE          OS_LAB    SystemCall.c      bin                  docs                 include           scripts  src

Child process completed.
root@Mani:~#
```

Child process completed.

REPORT: I executed the C program successfully. The program accepted a Linux command from the user, created a child process using `fork()`, and executed the command using the `exec()` system call. The parent process waited until the child process finished execution. Finally, the program displayed the process IDs of both the parent and child processes, and the message "Child process completed." was displayed.

Task-2: Using Linux terminal commands (`uname`, `lscpu`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Uname-a:

```
root@Mani:~# uname -a
Linux Mani 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@Mani:~#
```

REPORT: The `uname -a` command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.

Lscpu:

```
root@Mani:~# lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 42 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 14
On-line CPU(s) list: 0-13
Vendor ID: GenuineIntel
Model name: Intel(R) Core(TM) Ultra 5 225H
CPU family: 6
Model: 197
Thread(s) per core: 1
Core(s) per socket: 14
Socket(s): 1
Stepping: 2
BogoMIPS: 7372.80
Flags: fpu vme de pse tsc mtr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl vtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capabilities

Virtualization features:
Virtualization: VT-x
Hypervisor vendor: Microsoft
Virtualization type: full
Caches (sum of all):
L1d: 672 KiB (14 instances)
L1i: 896 KiB (14 instances)
L2: 42 MiB (14 instances)
L3: 18 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-13
Vulnerabilities:
Gather data sampling: Not affected
Ghostwrite: Not affected
Indirect target selection: Not affected
Itlb multihit: Not affected
Itlb: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Not affected
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE=IBRS SW sequence; BHI BHI_OIS_S
Srbds: Not affected
Taa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected
```

REPORT: The `lscpu` command displayed the CPU information of my system, such as the processor architecture, number of CPU cores, CPU model, and other hardware details. This helped me understand the processor resources available.

Ps:

```
root@Mani:~# ps
  PID TTY          TIME CMD
  430 pts/2        00:00:00 bash
  452 pts/2        00:00:00 ps
root@Mani:~#
```

REPORT: The ps command displayed the processes currently running in my terminal. I observed the process IDs and command names of the active processes.

Top:

```
root@Mani:~# top
top - 04:11:49 up 1 min, 1 user, load average: 0.04, 0.02, 0.00
Tasks: 27 total, 1 running, 26 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7658.2 total, 5984.2 free, 553.4 used, 1271.8 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7104.8 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
458	root	20	0	9340	5716	3496	R	0.3	0.1	0:00.01	top
1	root	20	0	21524	13104	9796	S	0.0	0.2	0:00.71	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.01	init-systemd(Ub
6	root	20	0	3196	2128	2008	S	0.0	0.0	0:00.00	init
55	root	19	-1	42248	16000	14820	S	0.0	0.2	0:00.31	systemd-journal
105	root	20	0	25404	6788	5124	S	0.0	0.1	0:00.16	systemd-udev
118	systemd+	20	0	21460	13160	10900	S	0.0	0.2	0:00.10	systemd-resolve
123	systemd+	20	0	91028	7976	7004	S	0.0	0.1	0:00.06	systemd-timesyn
183	root	20	0	4244	2680	2432	S	0.0	0.0	0:00.00	cron
184	message+	20	0	9596	5308	4632	S	0.0	0.1	0:00.06	dbus-daemon
192	root	20	0	17972	8884	7844	S	0.0	0.1	0:00.07	systemd-logind
194	root	20	0	1756104	13152	10888	S	0.0	0.2	0:00.09	wsl-pro-service
206	root	20	0	3124	1972	1836	S	0.0	0.0	0:00.00	agetty
213	syslog	20	0	222516	5940	4632	S	0.0	0.1	0:00.06	rsyslogd
230	root	20	0	107016	23204	13744	S	0.0	0.3	0:00.14	unattended-upgr
311	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00	SessionLeader
312	root	20	0	3200	1244	1108	S	0.0	0.0	0:00.00	Relay(313)
313	root	20	0	6080	5352	3676	S	0.0	0.1	0:00.02	bash
314	root	20	0	6668	4496	3732	S	0.0	0.1	0:00.01	login
359	root	20	0	20304	11600	9484	S	0.0	0.1	0:00.11	systemd
360	root	20	0	21160	3508	1796	S	0.0	0.0	0:00.00	(sd-pam)
380	root	20	0	6080	5292	3668	S	0.0	0.1	0:00.02	bash
425	root	20	0	3184	1116	980	S	0.0	0.0	0:00.00	SessionLeader
427	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.00	Relay(430)
430	root	20	0	6080	5348	3672	S	0.0	0.1	0:00.01	bash
459	root	20	0	25408	3920	2224	S	0.0	0.0	0:00.00	(udev-worker)
460	root	20	0	25408	3920	2224	S	0.0	0.0	0:00.00	(udev-worker)

REPORT: The top command displayed the live status of the system. I observed the CPU usage, memory usage, running processes, and system performance updating continuously.